

n8n in AI Resume Screening System

Role of the automation layer, its integration with Python, and the challenges solved

1. Why We Chose n8n

We chose n8n as the workflow automation layer because our project consists of multiple technologies that need to communicate with each other.

Our system uses Google Forms, Google Sheets, Google Drive, Python, AI processing, and the Streamlit dashboard. Connecting all these components manually would require additional integration code.

n8n provides a visual workflow in which each stage can be connected through nodes. This allowed us to focus on the actual resume-screening logic in Python while using n8n to manage the movement of data between different components.

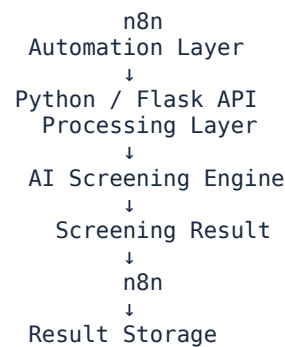
Another important reason for choosing n8n was its ability to work with HTTP APIs and binary files. Since the resume is uploaded as a PDF, the workflow needs to retrieve the actual file and send it to the Python backend. n8n provided the required workflow and file-handling capabilities.

2. Why We Used Python Alongside n8n

We did not use n8n as a replacement for Python because both technologies perform different roles.

n8n handles	Python handles
• Workflow automation • Data movement • Google Sheets/Drive integration • File retrieval • API communication • Structuring the workflow	• Resume processing • Backend logic • Communication with the screening engine • AI-based processing • Result generation • Result storage

Therefore, the architecture was divided into:



This separation made the system modular. Changes to the workflow could be made in n8n without rewriting the Python screening logic, while changes to the screening engine could be made in Python without rebuilding the entire workflow.

3. Overall n8n Workflow

The workflow developed for the project was approximately:

```
Google Form
↓
Google Sheets
↓
Resume Input
↓
Prepare Input
↓
Python Screening API
↓
Screening Result
↓
Parse / Structure
↓
Final Output
↓
Python Result API
↓
candidates.json
↓
Streamlit Dashboard
```

The workflow automatically transfers the candidate information and resume through the different stages.

4. How n8n Communicates With Python

The main communication between n8n and Python is done through HTTP requests.

The first HTTP request sends the resume and job description to the Python Flask API:

```
POST /screen
```

The request contains:

```
resume → PDF binary file
job_description → Job requirements
```

Python processes these inputs and returns a JSON screening result.

n8n then extracts the required fields and prepares the final output.

A second HTTP request sends the structured result back to Python:

```
POST /result
```

Python stores the result in:

```
data/candidates.json
```

Major Challenge 1

Sending the Actual PDF Instead of Just the Filename

One of the biggest challenges was handling the uploaded resume correctly.

Initially, the workflow was able to identify the resume filename, but the Python backend needed the actual PDF file for processing.

There is an important difference between:

```
Resume filename
```

and:

```
Resume binary data
```

Sending only:

```
resume.pdf
```

does not give Python access to the contents of the resume.

The HTTP Request node therefore had to be configured to send the binary data:

Body Content Type → Form-Data

resume
Type → n8n Binary File
Input Data Field Name → data

This allowed Python/Flask to receive the actual uploaded document through:

```
resume = request.files.get("resume")
```

This was a major integration step because the entire screening process depends on Python receiving the actual resume.

Major Challenge 2

Docker and Ubuntu Communication

Another major issue occurred because the two important components were running in different environments.

n8n → Docker container
Python → Ubuntu/WSL

Initially, it was assumed that Python could be accessed from n8n using:

```
http://localhost:5000
```

However, `localhost` inside the n8n Docker environment refers to the Docker environment itself, not the Ubuntu environment where Flask was running.

Therefore, n8n could not reach the Python server through localhost.

The solution was to expose the Flask application and use the Ubuntu/WSL IP address:

```
http://172.19.95.137:5000
```

The final connections became:

```
n8n
↓
http://172.19.95.137:5000/screen
↓
Python Flask
```

and:

```
n8n
↓
http://172.19.95.137:5000/result
↓
Python Flask
```

This was one of the most important networking problems solved during implementation.

Major Challenge 3

Maintaining the Correct Data Structure Between Nodes

Another challenge was that the output from one node had to become the correct input for the next node.

The Python screening response contained nested JSON information.

Instead of sending the entire response directly to the dashboard, n8n was used to extract and structure the required fields.

The important fields were:

```
candidate_name
match_score
key_strengths
missing_skills
final_verdict
explanation
success
```

The workflow therefore contained a Parse / Structure stage.

For example:

```
{{ $json.result.candidate_name }}
```

was used to extract the candidate name from the Python response.

This ensured that the final output had a consistent structure.

Major Challenge 4

File Reference vs Actual File

Another important challenge was understanding how Google Drive information moves through the workflow.

The Google Sheet contains a Drive URL/reference to the uploaded resume.

However, the Python API does not need just the Drive URL. It needs the actual PDF file.

Therefore, the workflow had to perform:

```
Google Sheet
  ↓
Drive File Reference
  ↓
Google Drive Download
  ↓
PDF Binary Data
  ↓
n8n
  ↓
Python
```

This distinction between a file reference and the actual binary file was essential for making the resume-processing workflow work.

Major Challenge 5

Preserving the Resume Binary Through the Workflow

While preparing candidate information, it was necessary to make sure that the resume binary was not accidentally removed when other fields were edited.

The workflow therefore preserved the input data while adding fields such as:

```
job_description
resume_file
```

The resume binary was then available to the HTTP Request node when the request was sent to Python.

This was important because an n8n workflow can successfully display a filename while still failing to pass the actual file.

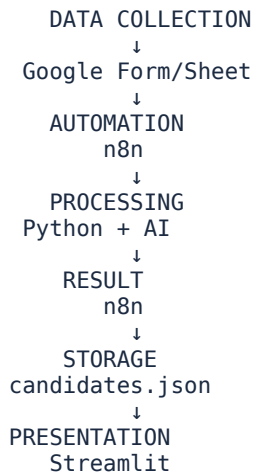
10. Why This Architecture Was Useful

Using n8n and Python together gave the project a modular architecture.

If we want to change how candidate information is collected, the n8n workflow can be modified without changing the Python screening engine.

Similarly, if we improve the Python AI screening logic, the n8n workflow does not need to be completely rebuilt as long as the API format remains compatible.

The architecture can therefore be represented as:



11. Future Scope of n8n in Our Project

The current n8n workflow can be extended in several ways.

Multiple Job Descriptions

HR could select different job descriptions and automatically screen candidates for different positions.

Automated Notifications

n8n could send an email notification to HR after a candidate has been processed.

Database Integration

The current JSON-based storage can eventually be replaced with a database while keeping n8n as the automation layer.

Error Handling

Dedicated error branches can be added to automatically detect failed resume processing and notify the administrator.

Recruitment Workflow Automation

The workflow could eventually be extended beyond initial screening to automate additional recruitment stages, such as interview scheduling and status notifications.

12. Conclusion

n8n was used as the central automation and integration layer of our AI Resume Screening System.

The combination of n8n and Python allowed us to separate workflow automation from resume-processing logic. n8n handled the movement of candidate data and files, while Python handled backend processing and AI-based screening.

The most significant challenges were not basic node configuration issues, but ensuring that the actual resume binary was transferred correctly, maintaining data structure between nodes, handling Google Drive file references, and establishing communication between n8n running in Docker and Python running in Ubuntu.

This integration demonstrated how workflow automation and programming can be combined to create a modular AI-based application.